基于下一代硬件的Ascend C SIMD与SIMT混合编程

作者：余启涛

时间：2026.3

背景与目标

课程背景：

- 下一代昇腾AI处理器架构持续演进，Vector计算单元新增支持SIMD和SIMT两种工作模式
- Ascend C 基于硬件架构提供SIMD与SIMT混合编程方式
- 适用融合算子或纯矢量算子开发，且矢量部分包含两种场景：
  1. SIMT：离散内存访问或复杂分支判断场景(如gather/scatter)
  2. SIMD：连续密集计算场景(如连续elementwise)

课程目标：

- 理解下一代芯片架构Vector单元核心升级点，SIMD与SIMT混合执行模式
- 理解Ascend C SIMD与SIMT混合编程模型，掌握核函数定义和调用方式
- 掌握SIMD与SIMT混合编程算子开发、精度调试和性能调试

目录

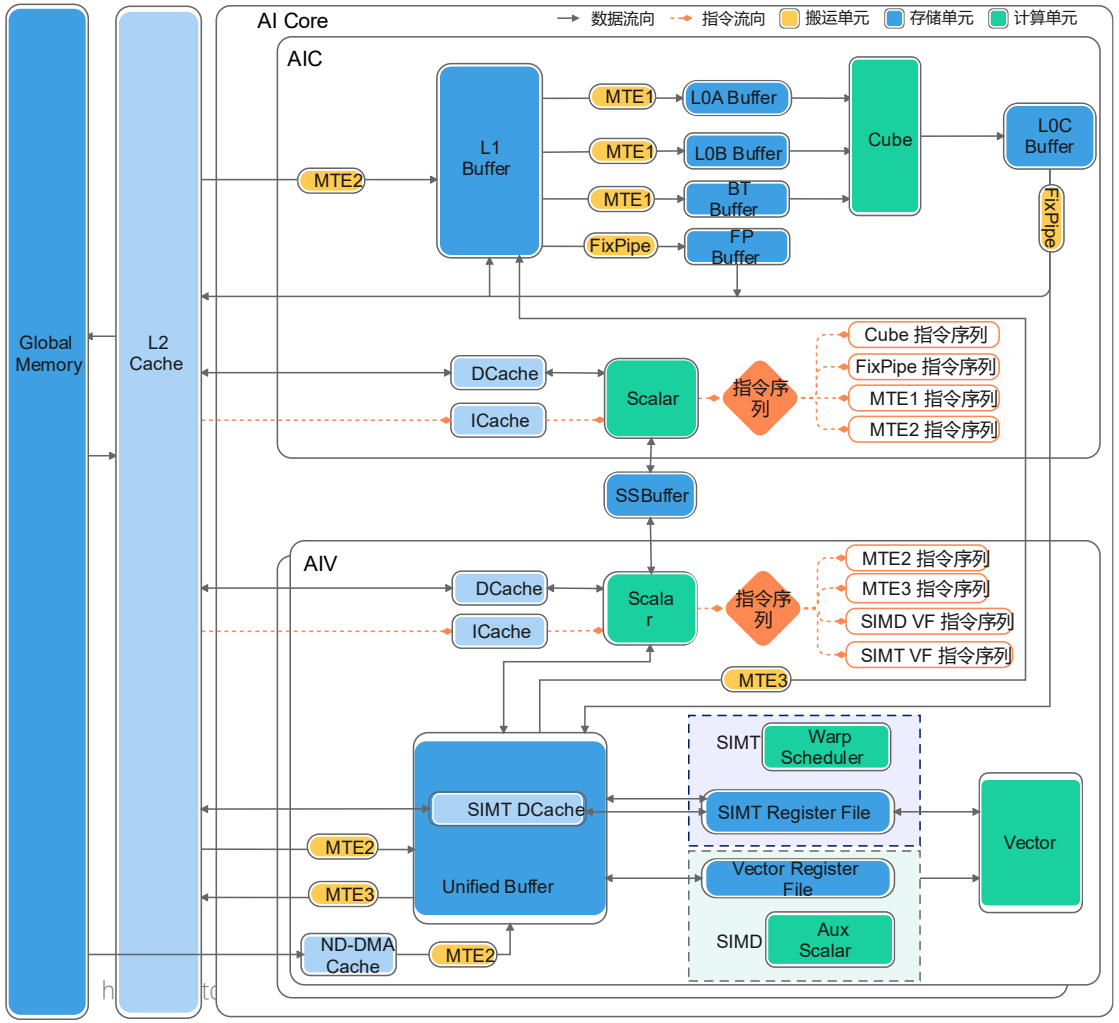
Part 1 SIMD与SIMT混合编程模型

Part 2 SIMD与SIMT混合算子示例

CANN

硬件架构概述

相比当代A2/A3，下一代AIV新增SIMT编程能力，可支持SIMD/SIMT混合编程



AIV:

- 可支持两种工作模式：SIMD和SIMT，以VF（ Vector Function ）为粒度进行切换
- **共享**
  - Vector ALU单元
  - ICache（Instruction Cache）
  - Unified Buffer
- **SIMT**
  - 每个AIV有4个Warp Scheduler, scheduler id = warp id % 4
  - 支持直接读写Global Memory或Unified Buffer (用作Shared Memory)
  - 最大支持**128KB** Data Cache，Data Cache直接复用Unified Buffer作为Cacheline（**128B**）
  - 通用寄存器**128KB**，每个寄存器**32bit**，每个线程可使用的寄存器受线程个数限制，具体如右表所示
  - 提供基于寄存器的vote，shuffle，reduce指令
- **SIMD**
  - 从A2/A3Memorybased架构切换到Regbased架构，新增SIMD Register File存储层级

线程个数	寄存器个数
2048	16
1024	32
512	64

AIC:

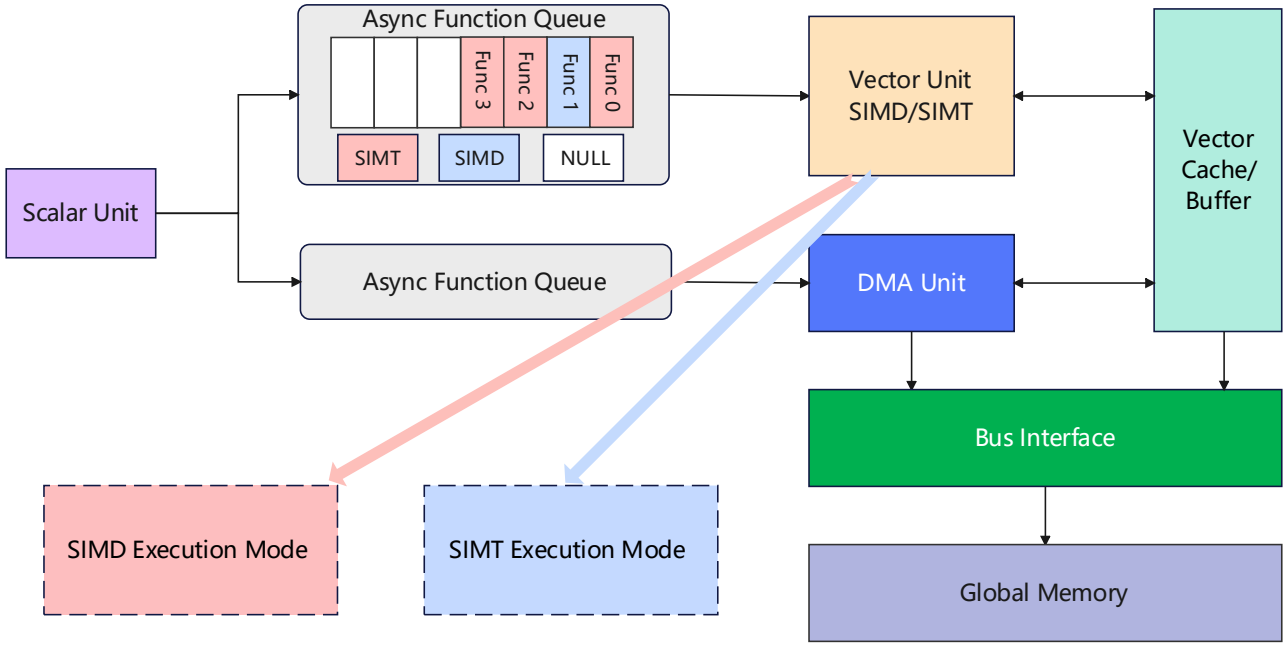
- 新增MX FP4/MX FP8类型
- 增强单核算力，增大核数
- 不再支持s4类型；不支持4:2稀疏矩阵

数据通路:

- 增加L0C Buffer -> Unified Buffer、Unified Buffer <-> L1 Buffer的数据通路
- 删除Global Memory -> L0A Buffer、Global Memory -> L0B Buffer的数据通路。删除L1 Buffer-> Global Memory的数据通路
- MTE2 数据搬运能力增强：新增NDDMA搬运、新增Compact模式、增强搬运维度
- SSBuffer，用于AIC和AIV的核间通信

CANN

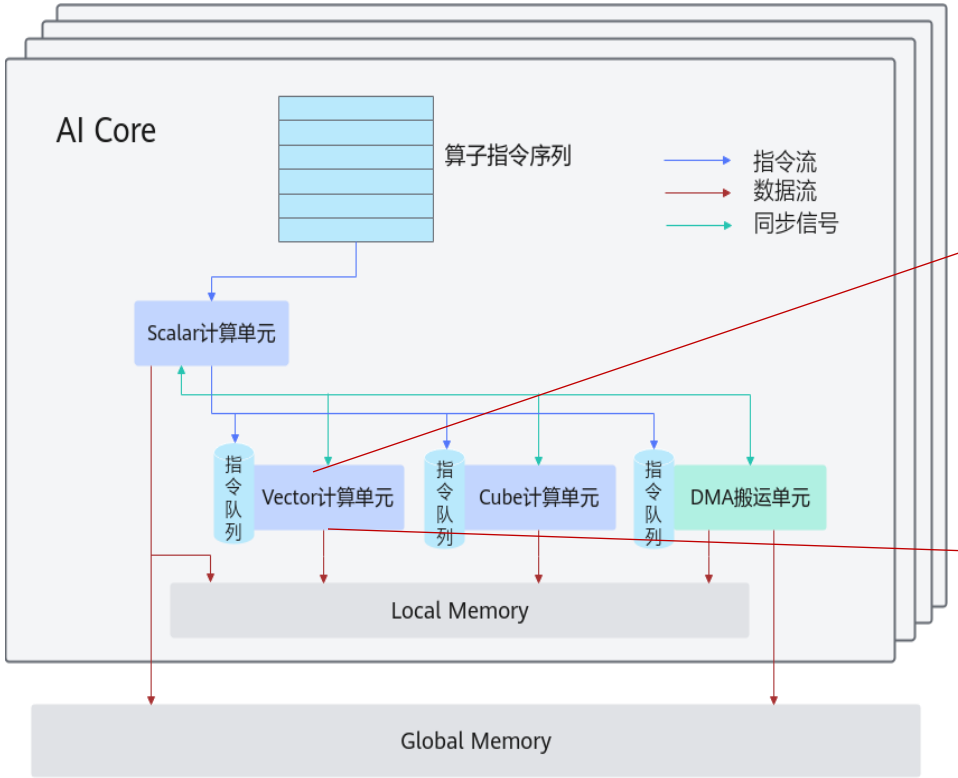
硬件架构: AIV SIMD & SIMT混合执行模式

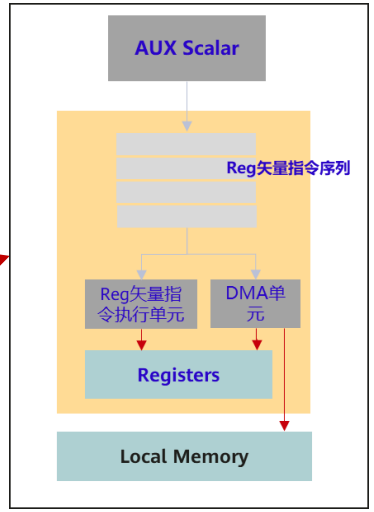


单个AIV执行逻辑:

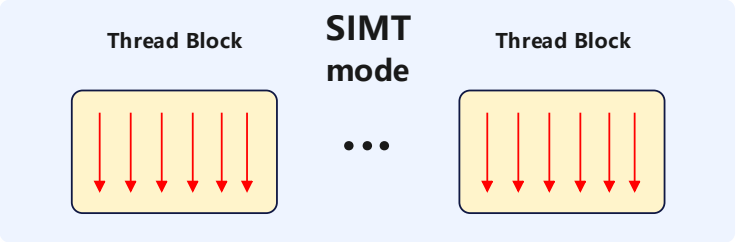
- 执行过程以 VF (Vector Function）为粒度
- Scalar发射VF到Vector Function Queue, VF相对于Scalar异步执行
- 可包含多个VF，每个VF可以选择使用SIMD或SIMT方式运行
- VF是串行执行的，同一时刻只有1个VF执行
- VF保证所有写操作在VF结束时完成

Ascend C SIMD & SIMT抽象硬件架构





SIMD mode



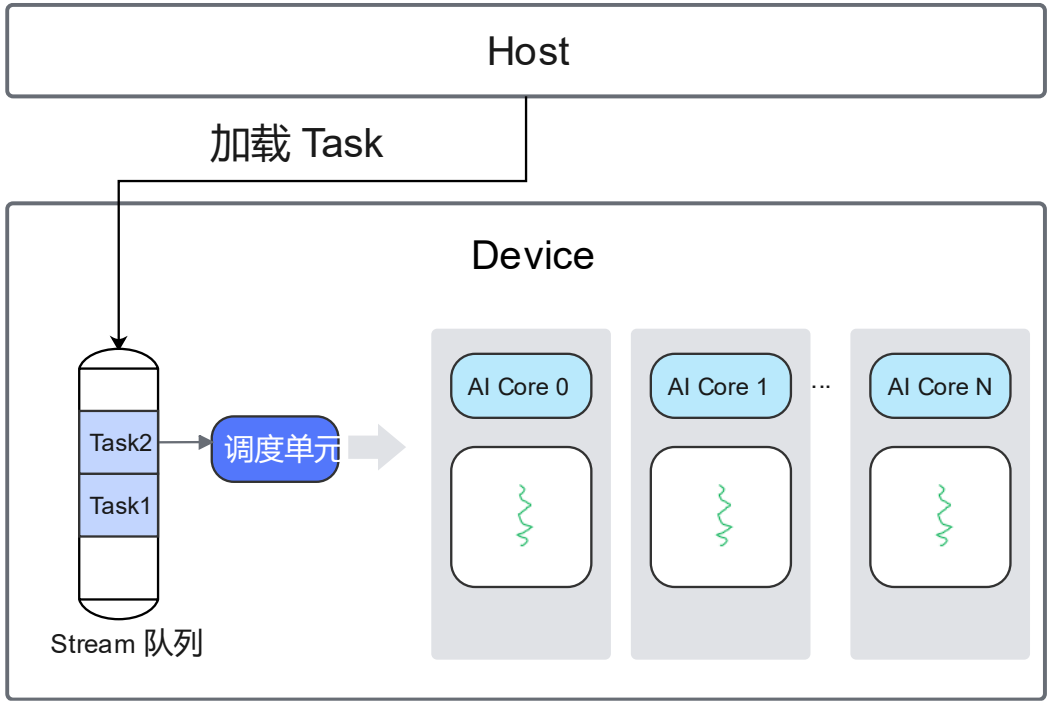
- AIC
  - 矩阵计算
- DMA
  - 数据搬运
- AIV SIMD mode
  - 新增Reg内存层级，开发者可以直接操作寄存器，更灵活，追求极致性能
  - 兼容当代芯片，支持基于Local Memory(UB）层级的计算接口

- AIV SIMT mode
  - 核心概念： Thread Block（线程块）、warp(调度线程组，32个线程)
  - 单个SIMT VF运行1个Thread Block，单个Thread Block最多支持2048个线程
  - 每个线程独立的寄存器和栈，用于存储局部变量，分配逻辑由编译器完成，对开发者透明
  - 直接读写Global Memory，Dcache对开发者透明
  - 可使用Unified Buffer作为Thread Block中所有线程的共享内存
  - 提供基于寄存器的vote，shuffle，reduce API

Ascend C SIMD & SIMT混合编程模型

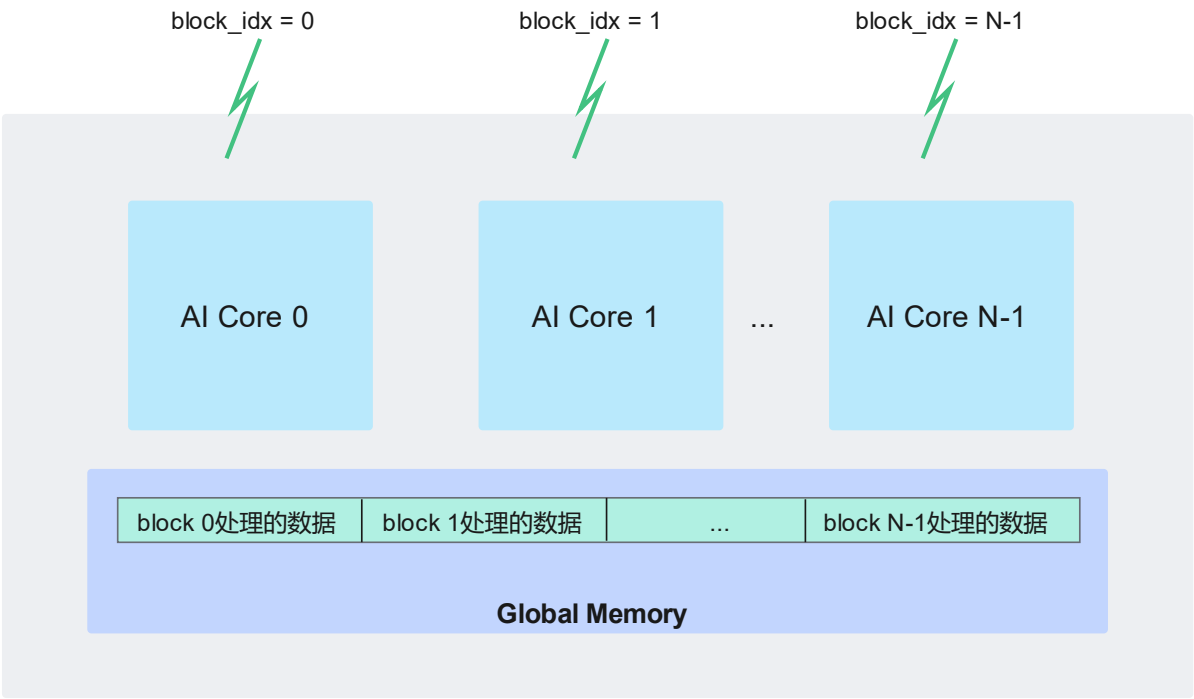
Host-Device异构协同机制

- Host侧：设备管理，运行时管理，存储管理，stream管理
- Device侧：计算密集任务



Device侧Kernel程序遵循SPMD编程模型

- Single-Program Multiple-Data
- 每个核处理不同的数据，程序使用内置变量block_id切分数据



CANN

Ascend C SIMD & SIMT混合编程： 核函数定义和调用

Kernel函数定义：使用__global__ __aicore__修饰

__global__ __aicore__ void custom_op_kernel(args...)

核函数中使用asc_vf_call调用SIMD和SIMT VF函数

Kernel调用方式：<<<>>>

kernel_name<<<numBlocks, dynUBufSize, stream>>>(args...)

- numBlocks：设置运行的核数
- dyUBufSize(默认为0)：设置动态内存大小，可通过以下方式使用：
  - ① 使用extern __ubuf__ half dynBuf[]动态数组分配
  - ② 通过Tque/Tpipe或LocalMemoryAllocator分配
- stream：类型为aclrtStream，用于维护异步操作的执行顺序

SIMD VF函数定义：使用__simd_vf__修饰

__simd_vf__ inline void simd_compute(...) { ... }

内置变量：block_num、block_idx等

SIMT VF函数定义：使用__simt_vf__修饰

__simt_vf__ __launch_bounds__(N) inline void simt_compute(...)

其中，__launch_bounds__(N)：指定SIMT VF启动的最大线程数

调用方式：asc_vf_call<simt_compute>(dim3(N), ...)

其中，dim3：指定线程块的3维层次结构

内置变量：gridDim、blockIdx、blockDim、threadIdx

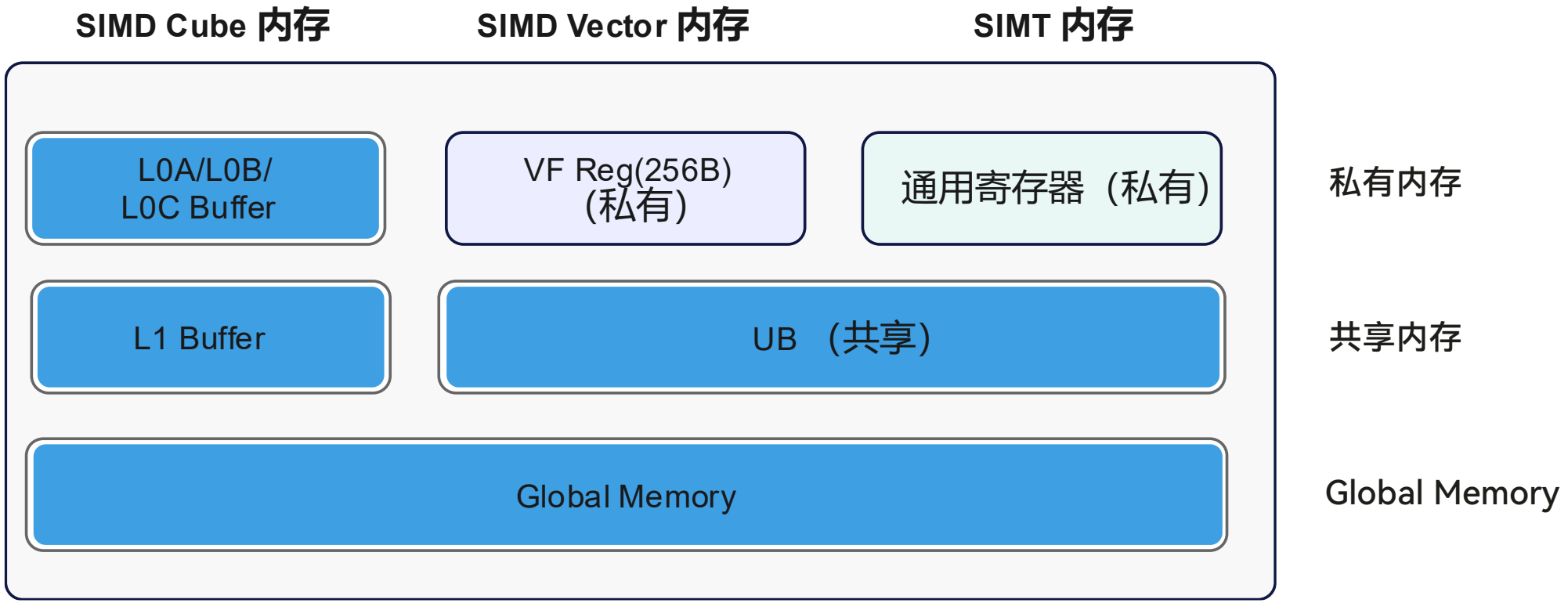
```
__global__ __aicore__ void custom_op_kernel(...)
{
   __ubuf__ float input_buf[INPUT_SIZE];
   asc_copy_gm2ub_align(...);
   ...
   asc_vf_call<simd_compute>(...);
    ...
   asc_vf_call<simt_compute>(dim3(THREAD_COUNT), ...);
   ...
}


uint32_t num_blocks = 64;
uint32_t dyn_ubuf_size = 0;
...
aclrtStream stream = nullptr;
aclrtCreateStream(&stream);
...
gather_kernel<<<num_blocks, dyn_ubuf_size, stream>>>(...);
...
```

```
__simd_vf__ inline void simd_compute(...) { ... }
```

```
__simt_vf__ __launch_bounds__(THREAD_COUNT) inline void simt_compute(...)
{
   int idx = blockIdx.x * blockDim.x + threadIdx.x;
   ...
}
```


CANN

Ascend C 内存层级



注：当代A2/A3支持GM/UB/L1/L0A/L0B/L0C，下一代新增VF Reg、SIMT通用寄存器(编译器自动分配)

CANN

Ascend C SIMD & SIMT混合编程：内存分配

Unified Buffer内存分配和申请方式

0

256k

静态内存

静态内存，使用数组分配
如__ubuf__ half staticBuf[1024]

动态内存

通过<<<>>>中dynUBufSize **指定的动态内存大小空间**，可通过以下方式申请使用：
① 使用extern __ubuf__ half dynBuf[]**动态数组**分配
② 使用Tque/Tpipe或LocalMemoryAllocator分配

注：为防止内存冲突，方式1和方式2动态内存方式不能同时使用

预留空间

编译器和ASC**预留空间**，大小为**8K**

Data Cache

剩余空间用作SIMT **Data Cache**，必须大于或等于**32K**
若不使用SIMT VF，只使用SIMD VF，则无需预留Data Cache

L1/L0A/L0B/L0C 内存申请方式

静态内存

静态内存，使用数组分配（待支持）
如__ubuf__ half staticBuf[1024]

动态内存

只支持使用以下方式申请使用：
① 使用Tque/Tpipe或LocalMemoryAllocator 分配

注：<<<>>> 参数仅指定UB动态内存申请，因此L1/L0A/L0B/L0C不支持通过数组分配动态内存

SIMD&SIMT混合编程场景，dynUBufSize必须配置，动态内存大小受指定大小限制，可同时使用静态数组分配内存

CANN

本章小结

- **下一代硬件架构AIV重大升级：**
  - 支持SIMD和SIMT两种模式，执行过程以 VF (Vector Function）为粒度切换
  - SIMD与SIMT共享ALU、ICache和UB
  - SIMD单元新增Register File存储层级，数据流变成GM<->UB<->Reg
  - 新增SIMT单元，支持将并行计算任务抽象成多线程方式执行
  - …

- **SIMD与SIMT混合编程模型：**
  - 总体遵循SPMD(Single-Program Multiple-Data）编程，每个Block运行相同代码处理不同数据
  - SIMD基于Register File提供RegTensor抽象数据结构和对应的计算API，提供更细粒度内存控制
  - SIMT抽象线程块概念，以多线程形式完成并行计算，开发者不感知Dcache和寄存器，编程易用性高
  - SIMD与SIMT共享Unified Buffer，高效协同
  - 核函数调用方法：kernel_name<<<numBlocks, dynUBufSize, stream>>>(args...)
  - 核函数内部使用asc_vf_call 调用SIMD VF或SIMT VF函数
  - SIMD VF使用__simd_vf__修饰，提供内置变量： block_num、block_idx等
  - SIMT VF使用__simt_vf__修饰，提供内置变量：gridDim、blockIdx、blockDim、threadIdx等

CANN

目录

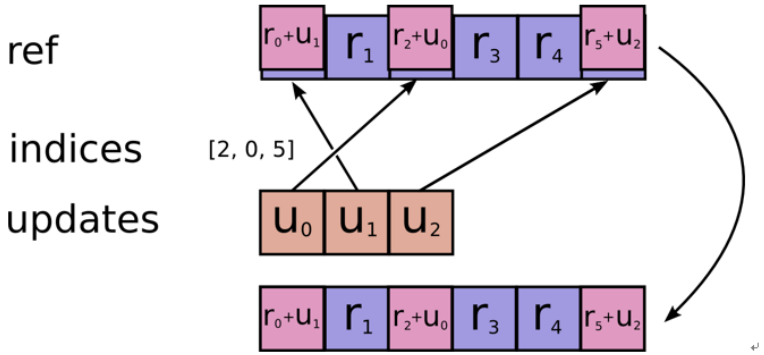
Part 1 SIMD与SIMT混合编程模型

Part 2 SIMD与SIMT混合算子示例

SIMD与SIMT混合编程示例： ScatterAdd算子

算子描述：
将 src 中的所有值按照 index 中指定的索引位置，累加到 self 中。
计算公式：

$$\mathrm{self}[\mathrm{index}[i][j][k]][j][k] \mathrel{+}= \mathrm{src}[i][j][k], \mathrm{dim} = 0$$
$$\mathrm{self}[i][\mathrm{index}[i][j][k]][k] \mathrel{+}= \mathrm{src}[i][j][k], \mathrm{dim} = 1$$
$$\mathrm{self}[i][i][\mathrm{index}[i][j][k]] \mathrel{+}= \mathrm{src}[i][j][k], \mathrm{dim} = 2$$




```
LocalTensor<IDX_T> indicesLocal = indicesInQueue_.DeQue<IDX_T>();
LocalTensor<VAR_T> updatesLocal = updatesInQueue_.DeQue<VAR_T>();

...
uint32_t uniqueIdNum = 0;
LocalTensor<CAST_T> indicesSortedLocal = sortIndicesQue_.Get<CAST_T>();
uniqueIdNum = SortAndComputeUniqueIdx<IDX_T>(
    indicesCount, indicesLocal, indicesSortedLocal, ...);
...


__local_mem__ CAST_T* indicesSortedPtr = (__local_mem__ CAST_T*)(indicesSortedLocal.GetPhyAddr()) + shiftOffset_;
__local_mem__ VAR_T* updatesLocalPtr = (__local_mem__ VAR_T*)(updatesLocal.GetPhyAddr());
uint32_t currentMaxThread = uniqueIdNum * totalCol >= THREAD_NUM_SORT ? THREAD_NUM_SORT : uniqueIdNum * totalCol;
int32_t threadBlock = currentMaxThread / totalCol;
threadBlock = threadBlock < uniqueIdNum ? threadBlock : uniqueIdNum;
Simt::VF_CALL<ScatterAddSimtSortCompute<IDX_T, VAR_T, CAST_T, ADDR_T, isUpdateScalar, scatterOp>>(
    Simt::Dim3({static_cast<uint32_t>(totalCol), static_cast<uint32_t>(threadBlock)}),
    varFirstDimSize, (__gm__ VAR_T*)(var_.GetPhyAddr()), updatesLocalPtr, indicesSortedPtr,
    (__local_mem__ uint32_t*)(updatesOriginIdxLocal.GetPhyAddr()),
    (__local_mem__ int32_t*)(uniqueIdCountLocal.GetPhyAddr()), uniqueIdNum, totalCol, updateScalarValue);
```

算子核心实现逻辑：

- 使用SIMD sort接口对index进行排序，获取同地址的索引
- 使用SIMT根据索引完成数据的离散scatter操作和add运算

示例算子实现已发布在CANN ops-nn仓：[算子实现]

CANN

SIMD与SIMT混合编程示例： gather算子

算子描述： 完整样例

从长度为8192的一维向量中获取指定索引的65536个数据。

计算公式：

$output_i = input_{index_i}, \quad index_i \in [0, len(input))$

	name	shape	data type	format
算子输入	input	8192	float	ND
	index	65536	uint32_t	ND
算子输出	output	65536	float	ND

特征：

- 离散内存访问，可使用SIMT快速实现
- 离散访问总数据量远小于（一个数量级）最大可用Unified Buffer空间

CANN

SIMD与SIMT混合编程示例： gather算子

离散内存访问场景：使用搬运指令将数据搬运到Unified Buffer提升性能，使用SIMT简化离散内存访问计算

```
__simt_vf__ __launch_bounds__(THREAD_COUNT) inline void simt_gather(
    __ubuf__ float* input, __gm__ uint32_t* index, __gm__ float* output)
{
    int32_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    ...
    uint32_t gather_idx = index[idx];
    ...
    output[idx] = input[gather_idx];
}

__global__ __vector__ void gather_kernel(__gm__ float* input,
    __gm__ uint32_t* index, __gm__ float* output)
{
    __ubuf__ float input_buf[INPUT_SIZE];
    uint32_t blk_length = INPUT_SIZE * sizeof(float);
    asc_copy_gm2ub_align(input_buf, input, 1, blk_length, 0, 0, false, 0, 0, 0);

    if ASC_IS_AIV {
        asc_sync_notify(PIPE_MTE2, PIPE_V, EVENT_ID0);
        asc_sync_wait(PIPE_MTE2, PIPE_V, EVENT_ID0);
    }

    asc_vf_call<simt_gather>(dim3(THREAD_COUNT), input_buf, index, output);
}

int32_t main(int32_t argc, char* argv[])
{
    ...
    uint32_t dyn_ubuf_size = 0;
    gather_kernel<<<num_blocks, dyn_ubuf_size, stream>>>(...);
    ...
}
```

SIMT VF入口函数：

- 计算线程待处理数据的离散索引值gather_idx
- 将Unified Buffer索引为gather_idx数据赋值到output

核函数入口

使用__ubuf__申请静态内存

使用搬运指令将数据将数据搬运到Unified Buffer

使用asc_vf_call调用SIMT VF

Host侧使用<<<>>>调用Kernel函数

CANN

SIMD与SIMT混合编程示例：其他场景

- 融合算子场景：Matmul(Cube计算) + LeakyRelu(Vector SIMT Mode)
- Vector算子：复杂分支判断(SIMT 编程) + 计算密集任务(SIMD Reg编程)
- …

SIMD与SIMT混合编程示例： 精度调试

使用类c函数printf打印算子计算过程数据，进行精度调试

```
__simt_vf__ __launch_bounds__(THREAD_COUNT) inline void simt_gather(
    __ubuf__ float* input, __gm__ uint32_t* index, __gm__ float* output)
{
    int32_t idx = blockIdx.x * blockDim.x + threadIdx.x;
    ...
    uint32_t gather_idx = index[idx];
    if (idx < 8) {
        printf("simt vf: thread idx: %d, gather idx: %d.\n", idx, gather_idx);
    }
    ...
    output[idx] = input[gather_idx];
}

__global__ __vector__ void gather_kernel(__gm__ float* input, __gm__ uint32_t* index, __gm__ float*
output)
{
    __ubuf__ float input_buf[INPUT_SIZE];
    uint32_t blk_length = INPUT_SIZE * sizeof(float);

    printf("kernel function: core id: %d, input length: %d.\n", block_idx, blk_length);

    asc_copy_gm2ub_align(input_buf, input, 1, blk_length, 0, 0, false, 0, 0, 0);
    ....
}
```

运行截图：

```
simt vf: thread idx: 7, gather idx: 117.
simt vf: thread idx: 6, gather idx: 1387.
simt vf: thread idx: 5, gather idx: 4530.
simt vf: thread idx: 4, gather idx: 1147.
simt vf: thread idx: 3, gather idx: 7586.
simt vf: thread idx: 2, gather idx: 2967.
simt vf: thread idx: 1, gather idx: 7347.
simt vf: thread idx: 0, gather idx: 2217.
Output: 89.4446 64.2083 3.53579 17.4963 56.4305 89.0181 66.864 71.7729
Golden: 89.4446 64.2083 3.53579 17.4963 56.4305 89.0181 66.864 71.7729
[Success] Case accuracy is verification passed.
```

SIMT VF中打印线程ID和对应的随机索引值

核函数中打印逻辑核ID和数据长度

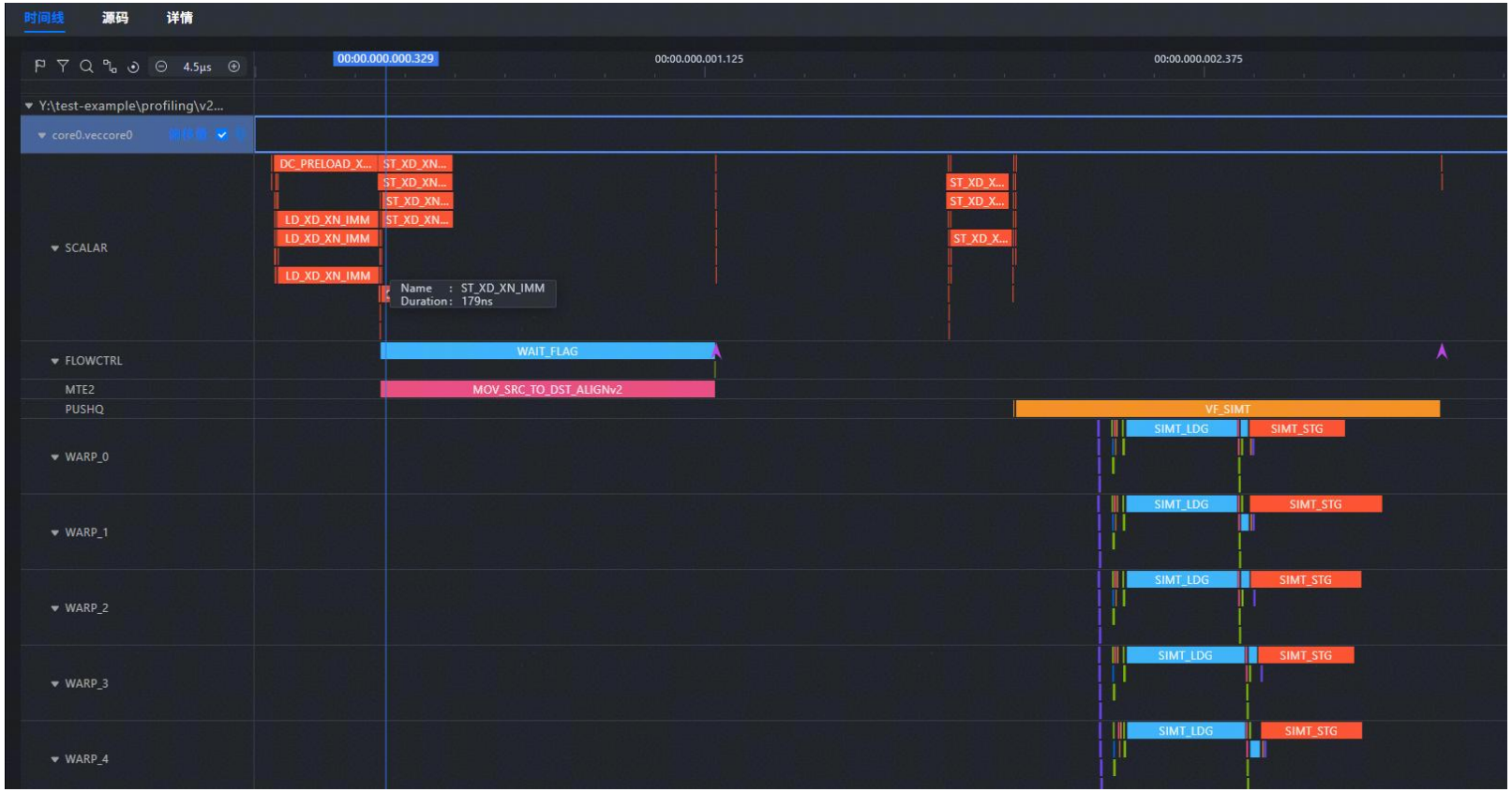
CANN

SIMD与SIMT混合编程示例： 性能调优(仿真)

在无硬件环境的场景下，通过编译仿真算子可执行文件，结合仿真器获取算子仿真性能数据，包含指令流水图等。

```
msprof op simulator ./demo
```

使用MindStudio Insight工具打开visualize_data.bin文件可视化查看性能数据



https://gitcode.com/cann

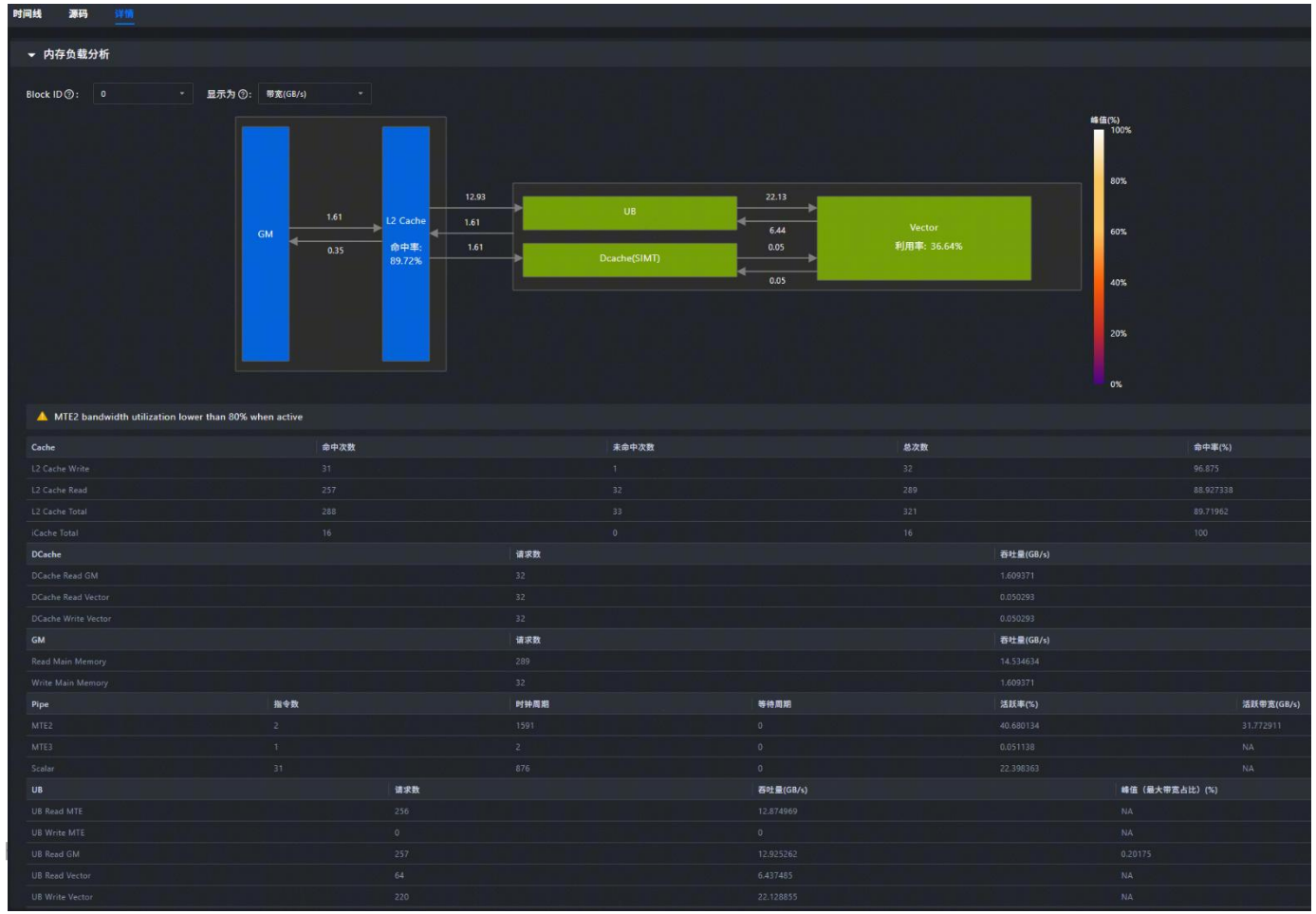
CANN

SIMD与SIMT混合编程示例： 性能调优(上板)

基于编译生成的可执行文件，直接在硬件上采集算子性能数据，包含算子基本信息和内存负载分析等。

```
msprof op ./demo
```

使用MindStudio Insight工具打开visualize_data.bin文件可视化查看性能数据



本章小结

- **通过算子样例介绍SIMD与SIMT混合编程开发方式：**
  - 介绍ScatterAdd算子功能和编程实现
  - 介绍gather算子功能和编程实现
  - 使用printf调试算子精度
  - 使用msprof工具采集仿真数据，通过指令流水图分析算子性能
  - 使用msprof工具采集上板数据，通过内存负载图分析算子性能
  - …

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN